# DATA STRUCTURES

## RECORD

### II/IV B. Tech, Semester-I

*Is Prepared By*


## MAILARU THARUN VINAY

AI&ML of Computer Science Engineering






**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES**

*(Permanently Affiliated to AU, Approved by AICTE, and Accredited by NBA &NAAC with an 'A' Grade)*

SANGIVALASA, Bheemili Mandal, VISAKHAPATNAM – 531162


2026-2027

# DATA STRUCTURES LABORATORY

## Record of C Programs

Aim • Description • Algorithm • Program • Output

## Index


| No. | Program |
|---|---|
| 1 | Singly Linked List – All Operations |
| 2 | Doubly Linked List – All Operations |
| 3 | Circular Doubly Linked List – All Operations |
| 4 | Stack Implementation Using Array |
| 5 | Stack Implementation Using Linked List |
| 6 | Queue Implementation Using Array |
| 7 | Queue Implementation Using Linked List |
| 8 | Priority Queue Using Array (ADT) |
| 9 | Deque (Double Ended Queue) Using Circular Array |
| 10 | Hashing – Collision Resolution Techniques |
| 11 | Binary Search Tree – Insertion, Deletion and Search |
| 12 | BST – Create, Traverse and Search (Week 11) |
| 13 | BST – Non-Recursive Inorder, Preorder and Postorder Traversals (Week 12) |
| 14 | Graph of N Cities – BFS and DFS (Program 13) |
| 15 | All-Pairs Shortest Paths – Warshall's / Floyd–Warshall (Program 14) |
| 16 | Dijkstra's Shortest Path – Start to End Node (Program 15) |

# Program 1: Singly Linked List – All Operations

## Aim

To design, develop and implement a menu-driven C program to perform insertion (at beginning, end and a given position), deletion (at beginning, end and a given position) and display operations on a singly linked list.

## Description

A singly linked list is a linear data structure made of nodes. Each node stores a data field and a pointer (next) to the following node; the last node points to NULL and a head pointer marks the start of the list. Memory is allocated dynamically with malloc() for every insertion and released with free() on deletion, so the list grows and shrinks at run time. Insertion and deletion at the beginning take O(1) time, whereas operations at the end or at a position need a traversal and take O(n) time.

## Algorithm

1. Start. Declare struct Node { data; next } and a global pointer head = NULL.
2. Insert at Beginning: create a new node, set newNode->next = head and head = newNode.
3. Insert at End: create a new node with next = NULL. If the list is empty make it head; otherwise traverse to the last node and set last->next = newNode.
4. Insert at Position: if pos == 1 insert at beginning; otherwise traverse to the node at pos-1 (report 'out of range' if NULL), then set newNode->next = temp->next and temp->next = newNode.
5. Delete at Beginning: if the list is empty report underflow; otherwise save temp = head, move head = head->next and free(temp).
6. Delete at End: if only one node, free it and set head = NULL; otherwise traverse to the second-last node, free its next node and set next = NULL.
7. Delete at Position: if pos == 1 delete the first node; otherwise traverse to pos-1, set toDelete = temp->next, temp->next = toDelete->next and free(toDelete).
8. Display: traverse from head to NULL printing each data value.
9. Repeat according to the user's menu choice until Exit is selected. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
};

struct Node *head = NULL;

/* ---------- INSERTION OPERATIONS ---------- */

void insertAtBeginning(int value) {
   struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->next = head;
   head = newNode;
   printf("Inserted %d at the beginning.\n", value);
}

void insertAtEnd(int value) {
   struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
```

```
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        head = newNode;
        printf("Inserted %d as the first node.\n", value);
        return;
    }

    struct Node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    printf("Inserted %d at the end.\n", value);
}

void insertAtPosition(int value, int pos) {
    if (pos < 1) {
        printf("Invalid position.\n");
        return;
    }

    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = value;

    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        printf("Inserted %d at position %d.\n", value, pos);
        return;
    }

    struct Node *temp = head;
    int count = 1;
    while (temp != NULL && count < pos - 1) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL) {
        printf("Position %d is out of range.\n", pos);
        free(newNode);
        return;
    }

    newNode->next = temp->next;
    temp->next = newNode;
    printf("Inserted %d at position %d.\n", value, pos);
}

/* ---------- DELETION OPERATIONS ---------- */

void deleteAtBeginning() {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }

    struct Node *temp = head;
    head = head->next;
```

```
    printf("Deleted %d from the beginning.\n", temp->data);
    free(temp);
}

void deleteAtEnd() {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }

    /* Case: only one node in the list */
    if (head->next == NULL) {
        printf("Deleted %d from the end.\n", head->data);
        free(head);
        head = NULL;
        return;
    }

    /* Traverse to the second-last node */
    struct Node *temp = head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }

    printf("Deleted %d from the end.\n", temp->next->data);
    free(temp->next);
    temp->next = NULL;
}

void deleteAtPosition(int pos) {
    if (head == NULL) {
        printf("List is empty. Nothing to delete.\n");
        return;
    }

    if (pos < 1) {
        printf("Invalid position.\n");
        return;
    }

    /* Case: delete first node */
    if (pos == 1) {
        struct Node *temp = head;
        head = head->next;
        printf("Deleted %d from position %d.\n", temp->data, pos);
        free(temp);
        return;
    }

    /* Traverse to the node just before the target position */
    struct Node *temp = head;
    int count = 1;
    while (temp != NULL && count < pos - 1) {
        temp = temp->next;
        count++;
    }

    /* Validate position */
    if (temp == NULL || temp->next == NULL) {
        printf("Position %d is out of range.\n", pos);
        return;
```

```
    }

    struct Node *toDelete = temp->next;
    temp->next = toDelete->next;
    printf("Deleted %d from position %d.\n", toDelete->data, pos);
    free(toDelete);
}

/* ---------- DISPLAY ---------- */

void display() {
    struct Node *temp = head;

    if (temp == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Linked List: ");
    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

/* ---------- MAIN MENU ---------- */

int main() {
    int choice, value, pos;

    do {
        printf("\n===== LINKED LIST OPERATIONS =====\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Position\n");
        printf("4. Delete at Beginning\n");
        printf("5. Delete at End\n");
        printf("6. Delete at Position\n");
        printf("7. Display\n");
        printf("8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                insertAtBeginning(value);
                break;

            case 2:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                insertAtEnd(value);
                break;

            case 3:
                printf("Enter value to insert: ");
                scanf("%d", &value);
                printf("Enter position: ");
```

```
            scanf("%d", &pos);
            insertAtPosition(value, pos);
            break;

        case 4:
            deleteAtBeginning();
            break;

        case 5:
            deleteAtEnd();
            break;

        case 6:
            printf("Enter position to delete: ");
            scanf("%d", &pos);
            deleteAtPosition(pos);
            break;

        case 7:
            display();
            break;

        case 8:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice. Try again.\n");
    }
  } while (choice != 8);

  return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
===== LINKED LIST OPERATIONS =====
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete at Beginning
5. Delete at End
6. Delete at Position
7. Display
8. Exit
Enter your choice: 1
Enter value to insert: 25
Inserted 25 at the beginning.
Enter your choice: 2
Enter value to insert: 40
Inserted 40 at the end.
Enter your choice: 3
Enter value to insert: 30
Enter position: 2
Inserted 30 at position 2.
Enter your choice: 7
Linked List: 25 -> 30 -> 40 -> NULL
Enter your choice: 6
Enter position to delete: 2
Deleted 30 from position 2.
Enter your choice: 7
Linked List: 25 -> 40 -> NULL
Enter your choice: 8
Exiting program.
Data Structures Lab
```

Dev-C++ console output | Sample run 1

# Program 2: Doubly Linked List – All Operations

## Aim

To design, develop and implement a menu-driven C program to perform insertion, deletion and forward/backward display operations on a doubly linked list.

## Description

In a doubly linked list every node holds three fields: data, a prev pointer to the previous node and a next pointer to the following node. This allows traversal in both directions and makes deletion of a known node easy because its predecessor is reachable directly. The price is one extra pointer per node and the need to update both prev and next links on every insertion and deletion.

## Algorithm

1. Start. Declare struct Node { data; prev; next } and a global head = NULL.
2. Insert at Beginning: newNode->prev = NULL, newNode->next = head; if head exists set head->prev = newNode; then head = newNode.
3. Insert at End: traverse to the last node, set last->next = newNode and newNode->prev = last.
4. Insert at Position: traverse to the node at pos-1, set newNode->next = temp->next, newNode->prev = temp; if temp->next exists set temp->next->prev = newNode; then temp->next = newNode.
5. Delete at Beginning: head = head->next; if head is not NULL set head->prev = NULL; free the old head.
6. Delete at End: traverse to the last node, set last->prev->next = NULL and free the last node (handle the single-node case separately).
7. Delete at Position: find the target node, link its prev and next nodes to each other (updating head if it was the first node) and free it.
8. Display Forward: traverse from head using next. Display Backward: go to the last node and traverse using prev.
9. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *prev;
   struct Node *next;
};

struct Node *head = NULL;

/* ---------- INSERTION OPERATIONS --------- */

void insertAtBeginning(int value) {
   struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->prev = NULL;
   newNode->next = head;

   if (head != NULL) {
      head->prev = newNode;
   }
```

```
    head = newNode;
    printf("Inserted %d at the beginning.\n", value);
}

void insertAtEnd(int value) {
    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;

    if (head == NULL) {
        newNode->prev = NULL;
        head = newNode;
        printf("Inserted %d as the first node.\n", value);
        return;
    }

    struct Node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
    printf("Inserted %d at the end.\n", value);
}

void insertAtPosition(int value, int pos) {
    if (pos < 1) {
        printf("Invalid position.\n");
        return;
    }

    if (pos == 1) {
        insertAtBeginning(value);
        return;
    }

    struct Node *temp = head;
    int count = 1;
    while (temp != NULL && count < pos - 1) {
        temp = temp->next;
        count++;
    }

    if (temp == NULL) {
        printf("Position %d is out of range.\n", pos);
        return;
    }

    struct Node *newNode = (struct Node *) malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = temp->next;
    newNode->prev = temp;

    if (temp->next != NULL) {
        temp->next->prev = newNode;
    }
    temp->next = newNode;
    printf("Inserted %d at position %d.\n", value, pos);
}

/* ---------- DELETION OPERATIONS ---------- */
```

```
void deleteAtBeginning() {
   if (head == NULL) {
      printf("List is empty. Nothing to delete.\n");
      return;
   }

   struct Node *temp = head;
   head = head->next;
   if (head != NULL) {
      head->prev = NULL;
   }
   printf("Deleted %d from the beginning.\n", temp->data);
   free(temp);
}

void deleteAtEnd() {
   if (head == NULL) {
      printf("List is empty. Nothing to delete.\n");
      return;
   }

   if (head->next == NULL) {
      printf("Deleted %d from the end.\n", head->data);
      free(head);
      head = NULL;
      return;
   }

   struct Node *temp = head;
   while (temp->next != NULL) {
      temp = temp->next;
   }

   printf("Deleted %d from the end.\n", temp->data);
   temp->prev->next = NULL;
   free(temp);
}

void deleteAtPosition(int pos) {
   if (head == NULL) {
      printf("List is empty. Nothing to delete.\n");
      return;
   }

   if (pos < 1) {
      printf("Invalid position.\n");
      return;
   }

   struct Node *temp = head;
   int count = 1;
   while (temp != NULL && count < pos) {
      temp = temp->next;
      count++;
   }

   if (temp == NULL) {
      printf("Position %d is out of range.\n", pos);
      return;
   }
```

```
    if (temp->prev != NULL) {
        temp->prev->next = temp->next;
    } else {
        head = temp->next;
    }

    if (temp->next != NULL) {
        temp->next->prev = temp->prev;
    }

    printf("Deleted %d from position %d.\n", temp->data, pos);
    free(temp);
}

/* ---------- DISPLAY ---------- */

void displayForward() {
    struct Node *temp = head;

    if (temp == NULL) {
        printf("List is empty.\n");
        return;
    }

    printf("Forward: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void displayBackward() {
    if (head == NULL) {
        printf("List is empty.\n");
        return;
    }

    struct Node *temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    printf("Backward: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}

/* ---------- MAIN MENU ---------- */

int main() {
    int choice, value, pos;

    do {
        printf("\n===== DOUBLY LINKED LIST OPERATIONS =====\n");
        printf("1. Insert at Beginning\n");
```

```
    printf("2. Insert at End\n");
    printf("3. Insert at Position\n");
    printf("4. Delete at Beginning\n");
    printf("5. Delete at End\n");
    printf("6. Delete at Position\n");
    printf("7. Display Forward\n");
    printf("8. Display Backward\n");
    printf("9. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            insertAtBeginning(value);
            break;

        case 2:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            insertAtEnd(value);
            break;

        case 3:
            printf("Enter value to insert: ");
            scanf("%d", &value);
            printf("Enter position: ");
            scanf("%d", &pos);
            insertAtPosition(value, pos);
            break;

        case 4:
            deleteAtBeginning();
            break;

        case 5:
            deleteAtEnd();
            break;

        case 6:
            printf("Enter position to delete: ");
            scanf("%d", &pos);
            deleteAtPosition(pos);
            break;

        case 7:
            displayForward();
            break;

        case 8:
            displayBackward();
            break;

        case 9:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice. Try again.\n");
    }
```

```
    } while (choice != 9);

    return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
===== DOUBLY LINKED LIST OPERATIONS =====
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete at Beginning
5. Delete at End
6. Delete at Position
7. Display Forward
8. Display Backward
9. Exit
Enter your choice: 1
Enter value to insert: 10
Inserted 10 at the beginning.
Enter your choice: 2
Enter value to insert: 30
Inserted 30 at the end.
Enter your choice: 3
Enter value to insert: 20
Enter position: 2
Inserted 20 at position 2.
Enter your choice: 7
Forward: 10 <-> 20 <-> 30 <-> NULL
Enter your choice: 8
Backward: 30 <-> 20 <-> 10 <-> NULL
Enter your choice: 5
Deleted 30 from the end.
Enter your choice: 9
Exiting program.
Data Structures Lab
```

Dev-C++ console output | Sample run 2

# Program 3: Circular Doubly Linked List – All Operations

## Aim

To design, develop and implement a menu-driven C program to perform insertion and deletion at the beginning, end and a given position, and to display a circular doubly linked list.

## Description

A circular doubly linked list is a doubly linked list in which the last node's next pointer points back to head and head's prev pointer points to the last node, so there is no NULL at either end. head->prev gives O(1) access to the last node, which makes insertion and deletion at both ends constant-time operations. Because there is no NULL to stop at, traversal uses a do-while loop that ends when the pointer returns to head.

## Algorithm

1. Start. Declare struct Node { data; next; prev } and head = NULL.
2. Insert at Beginning: if the list is empty the node points to itself and becomes head; otherwise link the new node between last (head->prev) and head and make it the new head.
3. Insert at End: same linking between last and head, but head is not changed.
4. Insert at Position: if pos == 1 call insert at beginning; otherwise move to the node at pos-1 and link the new node after it, fixing the next and prev pointers on both sides.
5. Delete at Beginning: if only one node, free it and set head = NULL; otherwise move head to head->next, relink it with last and free the old head.
6. Delete at End: if only one node, free it; otherwise let last = head->prev and secondLast = last->prev, link secondLast and head to each other and free last.
7. Delete at Position: if pos == 1 delete the first node; otherwise move to the node at pos, link its prev and next nodes together and free it.
8. Display: print data using a do-while loop from head until the pointer returns to head.
9. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *next;
   struct Node *prev;
};

struct Node *head = NULL;

void insertBeginning(int value) {
   struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   if (head == NULL) {
      newNode->next = newNode;
      newNode->prev = newNode;
      head = newNode;
      return;
   }
   struct Node *last = head->prev;
   newNode->next = head;
   newNode->prev = last;
```

```
    head->prev = newNode;
    last->next = newNode;
    head = newNode;
}

void insertEnd(int value) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    if (head == NULL) {
        newNode->next = newNode;
        newNode->prev = newNode;
        head = newNode;
        return;
    }
    struct Node *last = head->prev;
    newNode->next = head;
    newNode->prev = last;
    last->next = newNode;
    head->prev = newNode;
}

void insertPosition(int value, int pos) {
    if (pos == 1) {
        insertBeginning(value);
        return;
    }
    struct Node *temp = head;
    for (int i = 1; i < pos - 1; i++) {
        temp = temp->next;
    }
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = temp->next;
    newNode->prev = temp;
    temp->next->prev = newNode;
    temp->next = newNode;
}

void deleteBeginning() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    if (head->next == head) {
        free(head);
        head = NULL;
        return;
    }
    struct Node *last = head->prev;
    struct Node *temp = head;
    head = head->next;
    head->prev = last;
    last->next = head;
    free(temp);
}

void deleteEnd() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    if (head->next == head) {
```

```
        free(head);
        head = NULL;
        return;
    }
    struct Node *last = head->prev;
    struct Node *secondLast = last->prev;
    secondLast->next = head;
    head->prev = secondLast;
    free(last);
}

void deletePosition(int pos) {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    if (pos == 1) {
        deleteBeginning();
        return;
    }
    struct Node *temp = head;
    for (int i = 1; i < pos; i++) {
        temp = temp->next;
    }
    if (temp == head) {
        printf("Invalid position\n");
        return;
    }
    temp->prev->next = temp->next;
    temp->next->prev = temp->prev;
    free(temp);
}

void display() {
    if (head == NULL) {
        printf("List is empty\n");
        return;
    }
    struct Node *temp = head;
    printf("List: ");
    do {
        printf("%d ", temp->data);
        temp = temp->next;
    } while (temp != head);
    printf("\n");
}

int main() {
    int choice, value, pos;
    while (1) {
        printf("\n--- Circular Doubly Linked List Menu ---\n");
        printf("1. Insert at Beginning\n");
        printf("2. Insert at End\n");
        printf("3. Insert at Position\n");
        printf("4. Delete from Beginning\n");
        printf("5. Delete from End\n");
        printf("6. Delete from Position\n");
        printf("7. Display\n");
        printf("8. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
```

```
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insertBeginning(value);
                break;
            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insertEnd(value);
                break;
            case 3:
                printf("Enter value and position: ");
                scanf("%d %d", &value, &pos);
                insertPosition(value, pos);
                break;
            case 4:
                deleteBeginning();
                break;
            case 5:
                deleteEnd();
                break;
            case 6:
                printf("Enter position: ");
                scanf("%d", &pos);
                deletePosition(pos);
                break;
            case 7:
                display();
                break;
            case 8:
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
--- Circular Doubly Linked List Menu ---
1. Insert at Beginning
2. Insert at End
3. Insert at Position
4. Delete from Beginning
5. Delete from End
6. Delete from Position
7. Display
8. Exit
Enter your choice: 1
Enter value: 10
Enter your choice: 2
Enter value: 30
Enter your choice: 3
Enter value and position: 20 2
Enter your choice: 7
List: 10 20 30
Enter your choice: 6
Enter position: 2
Enter your choice: 7
List: 10 30
Enter your choice: 8
Data Structures Lab
```

Dev-C++ console output | Sample run 3

# Program 4: Stack Implementation Using Array

## Aim

To design, develop and implement a menu-driven C program to perform push, pop, peek and display operations on a stack using an array.

## Description

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle – the last element inserted is the first one removed. In the array implementation a fixed-size array stack[MAX] holds the elements and an integer top tracks the index of the topmost element (-1 for an empty stack). Overflow occurs when top == MAX-1 and underflow when top == -1. Push, pop and peek all run in O(1) time.

## Algorithm

1. Start. Declare an array stack[MAX] and set top = -1.
2. Push: if top == MAX-1 report Stack Overflow; otherwise increment top and store the value at stack[top].
3. Pop: if top == -1 report Stack Underflow; otherwise print stack[top] and decrement top.
4. Peek: if the stack is empty report it; otherwise print stack[top] without changing top.
5. Display: print the elements from index 0 to top (bottom to top).
6. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

int stack[MAX];
int top = -1;

/* check if stack is full */
int isFull() {
   return top == MAX - 1;
}

/* check if stack is empty */
int isEmpty() {
   return top == -1;
}

/* push an element onto the stack */
void push(int val) {
   if (isFull()) {
      printf("Stack Overflow\n");
      return;
   }
   stack[++top] = val;
   printf("Pushed %d\n", val);
}

/* pop an element from the stack */
void pop() {
   if (isEmpty()) {
      printf("Stack Underflow\n");
      return;
   }
```

```
    printf("Popped %d\n", stack[top--]);
}

/* view the top element without removing it */
void peek() {
    if (isEmpty()) {
        printf("Stack is empty\n");
        return;
    }
    printf("Top element: %d\n", stack[top]);
}

/* print all elements of the stack */
void display() {
    if (isEmpty()) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack (bottom -> top): ");
    for (int i = 0; i <= top; i++)
        printf("%d ", stack[i]);
    printf("\n");
}

int main() {
    int choice, val;
    while (1) {
        printf("\n1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &val);
                push(val);
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value to push: 10
Pushed 10
Enter choice: 1
Enter value to push: 20
Pushed 20
Enter choice: 1
Enter value to push: 30
Pushed 30
Enter choice: 4
Stack (bottom -> top): 10 20 30
Enter choice: 3
Top element: 30
Enter choice: 2
Popped 30
Enter choice: 4
Stack (bottom -> top): 10 20
Enter choice: 5
Data Structures Lab
```

Dev-C++ console output | Sample run 4

# Program 5: Stack Implementation Using Linked List

## Aim

To design, develop and implement a menu-driven C program to perform push, pop, peek and display operations on a stack using a linked list.

## Description

In the linked list implementation each stack element is a dynamically allocated node and a single pointer top refers to the topmost node (NULL for an empty stack). Because memory is allocated per push and freed per pop, there is no fixed size limit other than available system memory. Each element needs an extra pointer field, but all stack operations still run in O(1) time.

## Algorithm

1. Start. Declare struct Node { data; next } and a global pointer top = NULL.
2. Push: allocate a new node with malloc (report overflow if it fails), set newNode->next = top and top = newNode.
3. Pop: if top == NULL report Stack Underflow; otherwise save temp = top, print its data, move top = top->next and free(temp).
4. Peek: if the stack is empty report it; otherwise print top->data.
5. Display: traverse from top to NULL printing each data value (top to bottom).
6. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>

/* node structure for the linked list */
struct Node {
   int data;
   struct Node *next;
};

struct Node *top = NULL;

/* check if stack is empty */
int isEmpty() {
   return top == NULL;
}

/* push an element onto the stack */
void push(int val) {
   struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Stack Overflow (memory allocation failed)\n");
      return;
   }
   newNode->data = val;
   newNode->next = top;
   top = newNode;
   printf("Pushed %d\n", val);
}

/* pop an element from the stack */
void pop() {
```

```
    if (isEmpty()) {
        printf("Stack Underflow\n");
        return;
    }
    struct Node *temp = top;
    printf("Popped %d\n", temp->data);
    top = top->next;
    free(temp);
}

/* view the top element without removing it */
void peek() {
    if (isEmpty()) {
        printf("Stack is empty\n");
        return;
    }
    printf("Top element: %d\n", top->data);
}

/* print all elements of the stack */
void display() {
    if (isEmpty()) {
        printf("Stack is empty\n");
        return;
    }
    struct Node *ptr = top;
    printf("Stack (top -> bottom): ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

int main() {
    int choice, val;
    while (1) {
        printf("\n1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to push: ");
                scanf("%d", &val);
                push(val);
                break;
            case 2:
                pop();
                break;
            case 3:
                peek();
                break;
            case 4:
                display();
                break;
            case 5:
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
```

```
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value to push: 15
Pushed 15
Enter choice: 1
Enter value to push: 25
Pushed 25
Enter choice: 1
Enter value to push: 35
Pushed 35
Enter choice: 4
Stack (top -> bottom): 35 25 15
Enter choice: 3
Top element: 35
Enter choice: 2
Popped 35
Enter choice: 4
Stack (top -> bottom): 25 15
Enter choice: 5
Data Structures Lab
```

Dev-C++ console output | Sample run 5

# Program 6: Queue Implementation Using Array

## Aim

To design, develop and implement a menu-driven C program to perform enqueue, dequeue, peek and display operations on a linear queue using an array.

## Description

A queue is a linear data structure that follows the FIFO (First In, First Out) principle – the first element inserted is the first one removed. In the array implementation two indices, front and rear, are both initialised to -1. Elements are inserted at rear and removed from front. In this simple (linear) queue rear only moves forward, so once rear reaches MAX-1 the queue reports overflow even if slots at the front have been freed; a circular queue removes this limitation. Enqueue, dequeue and peek run in O(1) time.

## Algorithm

1. Start. Declare queue[MAX] and set front = rear = -1.
2. Enqueue: if rear == MAX-1 report Queue Overflow; otherwise if front == -1 set front = 0, then increment rear and store the value at queue[rear].
3. Dequeue: if the queue is empty (front == -1 or front > rear) report Queue Underflow; otherwise print queue[front] and increment front.
4. Peek: if the queue is empty report it; otherwise print queue[front].
5. Display: print the elements from index front to rear.
6. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

/* check if queue is full */
int isFull() {
   return rear == MAX - 1;
}

/* check if queue is empty */
int isEmpty() {
   return front == -1 || front > rear;
}

/* insert an element at the rear of the queue */
void enqueue(int val) {
   if (isFull()) {
      printf("Queue Overflow\n");
      return;
   }
   if (front == -1)
      front = 0;
   queue[++rear] = val;
   printf("Enqueued %d\n", val);
}
```

```
/* remove an element from the front of the queue */
void dequeue() {
   if (isEmpty()) {
      printf("Queue Underflow\n");
      return;
   }
   printf("Dequeued %d\n", queue[front]);
   front++;
}

/* view the front element without removing it */
void peek() {
   if (isEmpty()) {
      printf("Queue is empty\n");
      return;
   }
   printf("Front element: %d\n", queue[front]);
}

/* print all elements of the queue */
void display() {
   if (isEmpty()) {
      printf("Queue is empty\n");
      return;
   }
   printf("Queue (front -> rear): ");
   for (int i = front; i <= rear; i++)
      printf("%d ", queue[i]);
   printf("\n");
}

int main() {
   int choice, val;
   while (1) {
      printf("\n1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n");
      printf("Enter choice: ");
      scanf("%d", &choice);
      switch (choice) {
         case 1:
            printf("Enter value to enqueue: ");
            scanf("%d", &val);
            enqueue(val);
            break;
         case 2:
            dequeue();
            break;
         case 3:
            peek();
            break;
         case 4:
            display();
            break;
         case 5:
            exit(0);
         default:
            printf("Invalid choice\n");
      }
   }
   return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value to enqueue: 10
Enqueued 10
Enter choice: 1
Enter value to enqueue: 20
Enqueued 20
Enter choice: 1
Enter value to enqueue: 30
Enqueued 30
Enter choice: 4
Queue (front -> rear): 10 20 30
Enter choice: 3
Front element: 10
Enter choice: 2
Dequeued 10
Enter choice: 4
Queue (front -> rear): 20 30
Enter choice: 5
Data Structures Lab
```

Dev-C++ console output | Sample run 6

# Program 7: Queue Implementation Using Linked List

## Aim

To design, develop and implement a menu-driven C program to perform enqueue, dequeue, peek and display operations on a queue using a linked list.

## Description

The linked list queue keeps two pointers: front (head of the list, next element to be removed) and rear (tail of the list, last inserted element). Nodes are allocated on every enqueue and freed on every dequeue, so there is no fixed size and no false-overflow problem as in the linear array queue. When the last node is dequeued, rear must also be reset to NULL to avoid a dangling pointer. All operations take O(1) time.

## Algorithm

1. Start. Declare struct Node { data; next } and pointers front = rear = NULL.
2. Enqueue: allocate a new node with next = NULL. If the queue is empty set front = rear = newNode; otherwise set rear->next = newNode and rear = newNode.
3. Dequeue: if front == NULL report Queue Underflow; otherwise save temp = front, print its data, move front = front->next, set rear = NULL if the queue became empty, and free(temp).
4. Peek: if the queue is empty report it; otherwise print front->data.
5. Display: traverse from front to NULL printing each data value.
6. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>

/* node structure for the linked list */
struct Node {
   int data;
   struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

/* check if queue is empty */
int isEmpty() {
   return front == NULL;
}

/* insert an element at the rear of the queue */
void enqueue(int val) {
   struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
   if (newNode == NULL) {
      printf("Queue Overflow (memory allocation failed)\n");
      return;
   }
   newNode->data = val;
   newNode->next = NULL;
   if (isEmpty()) {
      front = newNode;
      rear = newNode;
   } else {
```

```
        rear->next = newNode;
        rear = newNode;
    }
    printf("Enqueued %d\n", val);
}

/* remove an element from the front of the queue */
void dequeue() {
    if (isEmpty()) {
        printf("Queue Underflow\n");
        return;
    }
    struct Node *temp = front;
    printf("Dequeued %d\n", temp->data);
    front = front->next;
    if (front == NULL)
        rear = NULL;   /* queue became empty */
    free(temp);
}

/* view the front element without removing it */
void peek() {
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }
    printf("Front element: %d\n", front->data);
}

/* print all elements of the queue */
void display() {
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }
    struct Node *ptr = front;
    printf("Queue (front -> rear): ");
    while (ptr != NULL) {
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
    printf("\n");
}

int main() {
    int choice, val;
    while (1) {
        printf("\n1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value to enqueue: ");
                scanf("%d", &val);
                enqueue(val);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                peek();
                break;
```

```
            case 4:
                display();
                break;
            case 5:
                exit(0);
            default:
                printf("Invalid choice\n");
        }
    }
    return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
1. Enqueue
2. Dequeue
3. Peek
4. Display
5. Exit
Enter choice: 1
Enter value to enqueue: 100
Enqueued 100
Enter choice: 1
Enter value to enqueue: 200
Enqueued 200
Enter choice: 1
Enter value to enqueue: 300
Enqueued 300
Enter choice: 4
Queue (front -> rear): 100 200 300
Enter choice: 3
Front element: 100
Enter choice: 2
Dequeued 100
Enter choice: 4
Queue (front -> rear): 200 300
Enter choice: 5
Data Structures Lab
```

# Program 8: Priority Queue Using Array (ADT)

## Aim

To design, develop and implement a C program to create a Priority Queue Abstract Data Type (ADT) that supports insertion, deletion of the highest-priority element and display, using an array as the underlying data structure.

## Description

A priority queue is an abstract data type in which every element has an associated priority and elements are served according to priority rather than arrival order. Here each element is a structure holding data and priority, and the element with the smallest priority number is treated as the highest priority. Insertion simply appends the element (O(1)); deletion scans the array for the minimum priority, removes it and shifts the remaining elements left (O(n)). Priority queues are used in CPU scheduling, Dijkstra's and Prim's algorithms and simulations.

## Algorithm

1. Start. Declare an array pq[MAX] of structures (data, priority) and set size = 0.
2. Enqueue: if size == MAX report that the queue is full; otherwise store the (data, priority) pair at pq[size] and increment size.
3. Dequeue: if size == 0 report that the queue is empty; otherwise scan the array to find the index of the element with the smallest priority number, print it, shift all following elements one position left and decrement size.
4. Display: if the queue is empty report it; otherwise print data and priority of every element from index 0 to size-1.
5. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

struct PriorityQueue {
   int data;
   int priority;
};

struct PriorityQueue pq[MAX];
int size = 0;

// Insert an element
void enqueue(int data, int priority) {
   if (size == MAX) {
      printf("Priority Queue is full!\n");
      return;
   }
   pq[size].data = data;
   pq[size].priority = priority;
   size++;
   printf("Element inserted successfully.\n");
}

// Delete the highest-priority element
void dequeue() {
   if (size == 0) {
```

```
        printf("Priority Queue is empty!\n");
        return;
    }
    int highest = 0;
    // Find element with highest priority
    for (int i = 1; i < size; i++) {
        if (pq[i].priority < pq[highest].priority) {
            highest = i;
        }
    }
    printf("Deleted element: %d\n", pq[highest].data);
    // Shift remaining elements
    for (int i = highest; i < size - 1; i++) {
        pq[i] = pq[i + 1];
    }
    size--;
}

// Display the priority queue
void display() {
    if (size == 0) {
        printf("Priority Queue is empty!\n");
        return;
    }
    printf("\nPriority Queue:\n");
    printf("Data\tPriority\n");
    for (int i = 0; i < size; i++) {
        printf("%d\t%d\n", pq[i].data, pq[i].priority);
    }
}

int main() {
    int choice, data, priority;
    while (1) {
        printf("\n--- Priority Queue ---\n");
        printf("1. Enqueue\n");
        printf("2. Dequeue\n");
        printf("3. Display\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter data: ");
                scanf("%d", &data);
                printf("Enter priority (smaller number = higher priority): ");
                scanf("%d", &priority);
                enqueue(data, priority);
                break;
            case 2:
                dequeue();
                break;
            case 3:
                display();
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
```

```
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
--- Priority Queue ---
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter data: 100
Enter priority (smaller number = higher priority): 3
Element inserted successfully.
Enter your choice: 1
Enter data: 200
Enter priority (smaller number = higher priority): 1
Element inserted successfully.
Enter your choice: 1
Enter data: 300
Enter priority (smaller number = higher priority): 2
Element inserted successfully.
Enter your choice: 3
Priority Queue:
Data    Priority
100     3
200     1
300     2
Enter your choice: 2
Deleted element: 200
Enter your choice: 3
Priority Queue:
Data    Priority
100     3
300     2
Enter your choice: 4
Data Structures Lab
```

# Program 9: Deque (Double Ended Queue) Using Circular Array

## Aim

To design, develop and implement a menu-driven C program to perform insertion and deletion at both ends of a double ended queue (deque) implemented using a circular array.

## Description

A deque allows insertion and deletion at both the front and the rear ends, so it supports four operations: insert front, insert rear, delete front and delete rear. Using a circular array avoids the wastage of space seen in a linear array: when front or rear reaches one end of the array it wraps around to the other end when space is available. All operations take O(1) time.

## Algorithm

1. Start. Declare deque[MAX] and set front = rear = -1.
2. isFull: the deque is full if (front == 0 and rear == MAX-1) or front == rear+1. isEmpty: front == -1.
3. Insert Front: if full report overflow; if empty set front = rear = 0; else if front == 0 wrap front to MAX-1; else decrement front. Store the value at deque[front].
4. Insert Rear: if full report overflow; if empty set front = rear = 0; else if rear == MAX-1 wrap rear to 0; else increment rear. Store the value at deque[rear].
5. Delete Front: if empty report underflow; otherwise print deque[front]; if front == rear reset both to -1, else if front == MAX-1 wrap to 0, else increment front.
6. Delete Rear: if empty report underflow; otherwise print deque[rear]; if front == rear reset both to -1, else if rear == 0 wrap to MAX-1, else decrement rear.
7. Display: starting at front, print elements moving circularly with (i+1) % MAX until rear is reached.
8. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#define MAX 5

int deque[MAX];
int front = -1, rear = -1;

// Check if deque is full
int isFull() {
   return ((front == 0 && rear == MAX - 1) || (front == rear + 1));
}

// Check if deque is empty
int isEmpty() {
   return (front == -1);
}

// Insert element at the front
void insertFront(int value) {
   if (isFull()) {
      printf("Deque is full! Cannot insert.\n");
      return;
   }
   if (front == -1) {         // first element
      front = 0;
      rear = 0;
   } else if (front == 0) {
      front = MAX - 1;       // wrap around
```

```
    } else {
        front = front - 1;
    }
    deque[front] = value;
    printf("Inserted %d at front.\n", value);
}

// Insert element at the rear
void insertRear(int value) {
    if (isFull()) {
        printf("Deque is full! Cannot insert.\n");
        return;
    }
    if (front == -1) {          // first element
        front = 0;
        rear = 0;
    } else if (rear == MAX - 1) {
        rear = 0;               // wrap around
    } else {
        rear = rear + 1;
    }
    deque[rear] = value;
    printf("Inserted %d at rear.\n", value);
}

// Delete element from the front
void deleteFront() {
    if (isEmpty()) {
        printf("Deque is empty! Cannot delete.\n");
        return;
    }
    printf("Deleted %d from front.\n", deque[front]);
    if (front == rear) {        // only one element
        front = -1;
        rear = -1;
    } else if (front == MAX - 1) {
        front = 0;              // wrap around
    } else {
        front = front + 1;
    }
}

// Delete element from the rear
void deleteRear() {
    if (isEmpty()) {
        printf("Deque is empty! Cannot delete.\n");
        return;
    }
    printf("Deleted %d from rear.\n", deque[rear]);
    if (front == rear) {        // only one element
        front = -1;
        rear = -1;
    } else if (rear == 0) {
        rear = MAX - 1;         // wrap around
    } else {
        rear = rear - 1;
    }
}

// Display all elements of the deque
void display() {
    if (isEmpty()) {
```

```
        printf("Deque is empty!\n");
        return;
    }
    printf("Deque elements: ");
    int i = front;
    while (1) {
        printf("%d ", deque[i]);
        if (i == rear)
            break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}

int main() {
    int choice, value;
    while (1) {
        printf("\n----- DEQUE MENU (Circular Array) -----\n");
        printf("1. Insert at Front\n");
        printf("2. Insert at Rear\n");
        printf("3. Delete from Front\n");
        printf("4. Delete from Rear\n");
        printf("5. Display\n");
        printf("6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                insertFront(value);
                break;
            case 2:
                printf("Enter value: ");
                scanf("%d", &value);
                insertRear(value);
                break;
            case 3:
                deleteFront();
                break;
            case 4:
                deleteRear();
                break;
            case 5:
                display();
                break;
            case 6:
                printf("Exiting program.\n");
                return 0;
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
----- DEQUE MENU (Circular Array) -----
1. Insert at Front
2. Insert at Rear
3. Delete from Front
4. Delete from Rear
5. Display
6. Exit
Enter your choice: 2
Enter value: 10
Inserted 10 at rear.
Enter your choice: 2
Enter value: 20
Inserted 20 at rear.
Enter your choice: 1
Enter value: 5
Inserted 5 at front.
Enter your choice: 5
Deque elements: 5 10 20
Enter your choice: 3
Deleted 5 from front.
Enter your choice: 5
Deque elements: 10 20
Enter your choice: 6
Exiting program.
Data Structures Lab
```

# Program 10: Hashing – Collision Resolution Techniques

## Aim

To design, develop and implement a C program to handle collisions in a hash table using (a) linear probing, (b) quadratic probing and (c) separate chaining.

## Description

Hashing maps a key to an index of an array (hash table) using a hash function, here h(key) = key % TABLE_SIZE. A collision occurs when two different keys map to the same index. Linear probing tries index, index+1, index+2, ... (mod TABLE_SIZE) and can cause primary clustering. Quadratic probing tries index + $i^2$ and reduces primary clustering. Separate chaining keeps a linked list at every slot, so colliding keys are simply appended to the chain at that index.

## Algorithm

1. Start. Fix TABLE_SIZE and define hashFunction(key) = key % TABLE_SIZE.
2. Linear probing: initialise all slots to -1; compute index = hash(key); for i = 0 to TABLE_SIZE-1 compute probe = (index + i) % TABLE_SIZE and place the key in the first empty slot; if none is found report that the table is full.
3. Quadratic probing: initialise all slots to -1; compute index = hash(key); for i = 0 to TABLE_SIZE-1 compute probe = (index + i*i) % TABLE_SIZE and place the key in the first empty slot; otherwise report that no free slot exists.
4. Separate chaining: initialise every slot to NULL; compute index = hash(key); create a new node; if the slot is empty make it the head, otherwise traverse to the end of the chain and append it.
5. Display each table (or chains) after the insertions.
6. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>
#define TABLE_SIZE 10

/* ---------- Tables for the three techniques ---------- */
int linearTable[TABLE_SIZE];
int quadraticTable[TABLE_SIZE];

struct Node {
   int key;
   struct Node *next;
};
struct Node *chainTable[TABLE_SIZE];

/* ---------- Hash function (common to all techniques) ---------- */
int hashFunction(int key) {
   return key % TABLE_SIZE;
}

/* ================= LINEAR PROBING ================= */
void initLinear() {
   for (int i = 0; i < TABLE_SIZE; i++)
      linearTable[i] = -1;   /* -1 marks an empty slot */
}

void insertLinear(int key) {
   int index = hashFunction(key);
```

```
    int i;
    for (i = 0; i < TABLE_SIZE; i++) {
        int probe = (index + i) % TABLE_SIZE;   /* linear probe: h(k)+i */
        if (linearTable[probe] == -1) {
            linearTable[probe] = key;
            printf("Key %d inserted at index %d (linear probing).\n", key, probe);
            return;
        }
    }
    printf("Hash table is full! Cannot insert key %d.\n", key);
}

void displayLinear() {
    printf("\nLinear Probing Table:\n");
    printf("Index\tValue\n");
    for (int i = 0; i < TABLE_SIZE; i++) {
        if (linearTable[i] == -1)
            printf("%d\t-\n", i);
        else
            printf("%d\t%d\n", i, linearTable[i]);
    }
}

/* ================= QUADRATIC PROBING ================= */
void initQuadratic() {
    for (int i = 0; i < TABLE_SIZE; i++)
        quadraticTable[i] = -1;
}

void insertQuadratic(int key) {
    int index = hashFunction(key);
    int i;
    for (i = 0; i < TABLE_SIZE; i++) {
        int probe = (index + i * i) % TABLE_SIZE;   /* quadratic probe: h(k)+i*i */
        if (quadraticTable[probe] == -1) {
            quadraticTable[probe] = key;
            printf("Key %d inserted at index %d (quadratic probing).\n", key, probe);
            return;
        }
    }
    printf("Hash table is full or no free slot found for key %d.\n", key);
}

void displayQuadratic() {
    printf("\nQuadratic Probing Table:\n");
    printf("Index\tValue\n");
    for (int i = 0; i < TABLE_SIZE; i++) {
        if (quadraticTable[i] == -1)
            printf("%d\t-\n", i);
        else
            printf("%d\t%d\n", i, quadraticTable[i]);
    }
}

/* ================= SEPARATE CHAINING ================= */
void initChaining() {
    for (int i = 0; i < TABLE_SIZE; i++)
        chainTable[i] = NULL;
}

void insertChaining(int key) {
```

```
    int index = hashFunction(key);
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->key = key;
    newNode->next = NULL;

    if (chainTable[index] == NULL) {
        chainTable[index] = newNode;
    } else {
        struct Node *temp = chainTable[index];
        while (temp->next != NULL)
            temp = temp->next;
        temp->next = newNode;
    }
    printf("Key %d inserted at index %d (separate chaining).\n", key, index);
}

void displayChaining() {
    printf("\nSeparate Chaining Table:\n");
    printf("Index\tChain\n");
    for (int i = 0; i < TABLE_SIZE; i++) {
        printf("%d\t", i);
        struct Node *temp = chainTable[i];
        if (temp == NULL) {
            printf("-");
        } else {
            while (temp != NULL) {
                printf("%d -> ", temp->key);
                temp = temp->next;
            }
            printf("NULL");
        }
        printf("\n");
    }
}

/* ================= DRIVER PROGRAM ================= */
int main() {
    int choice, key;

    initLinear();
    initQuadratic();
    initChaining();

    while (1) {
        printf("\n----- HASHING - COLLISION RESOLUTION TECHNIQUES -----\n");
        printf("1. Insert using Linear Probing\n");
        printf("2. Insert using Quadratic Probing\n");
        printf("3. Insert using Separate Chaining\n");
        printf("4. Display Linear Probing Table\n");
        printf("5. Display Quadratic Probing Table\n");
        printf("6. Display Separate Chaining Table\n");
        printf("7. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter key to insert: ");
                scanf("%d", &key);
                insertLinear(key);
                break;
```

```
            case 2:
                printf("Enter key to insert: ");
                scanf("%d", &key);
                insertQuadratic(key);
                break;
            case 3:
                printf("Enter key to insert: ");
                scanf("%d", &key);
                insertChaining(key);
                break;
            case 4:
                displayLinear();
                break;
            case 5:
                displayQuadratic();
                break;
            case 6:
                displayChaining();
                break;
            case 7:
                printf("Exiting program.\n");
                exit(0);
            default:
                printf("Invalid choice! Try again.\n");
        }
    }
    return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
----- HASHING - COLLISION RESOLUTION TECHNIQUES -----
1. Insert using Linear Probing
2. Insert using Quadratic Probing
3. Insert using Separate Chaining
4. Display Linear Probing Table
5. Display Quadratic Probing Table
6. Display Separate Chaining Table
7. Exit
Enter your choice: 1
Enter key to insert: 23
Key 23 inserted at index 3 (linear probing).
Enter your choice: 1
Enter key to insert: 43
Key 43 inserted at index 4 (linear probing).
Enter your choice: 2
Enter key to insert: 23
Key 23 inserted at index 3 (quadratic probing).
Enter your choice: 2
Enter key to insert: 43
Key 43 inserted at index 4 (quadratic probing).
Enter your choice: 3
Enter key to insert: 23
Key 23 inserted at index 3 (separate chaining).
Enter your choice: 3
Enter key to insert: 43
Key 43 inserted at index 3 (separate chaining).
Enter your choice: 6
Separate Chaining Table:
Index  Chain
0      -
1      -
2      -
3      23 -> 43 -> NULL
4      -
...
Enter your choice: 7
Exiting program.
Data Structures Lab
```

Dev-C++ console output | Sample run 10

# Program 11: Binary Search Tree – Insertion, Deletion and Search

## Aim

To write a C program to perform insertion, deletion and search of an element in a Binary Search Tree (BST).

## Description

A Binary Search Tree is a binary tree in which, for every node, all values in the left subtree are smaller and all values in the right subtree are larger than the node's value. This ordering gives O(log n) insertion, deletion and search for a balanced tree and O(n) in the worst case of a skewed tree. Deletion has three cases: a leaf node is removed directly; a node with one child is replaced by that child; a node with two children takes the value of its inorder successor, which is then deleted from the right subtree. An inorder traversal prints the keys in sorted order.

## Algorithm

1. Start. Declare struct Node { data; left; right } and root = NULL.
2. Insert(root, key): if root is NULL create a new node; if key < root->data insert into the left subtree; if key > root->data insert into the right subtree; ignore duplicates; return root.
3. Search(root, key): if root is NULL or root->data == key return root; if key < root->data search the left subtree, else search the right subtree.
4. Delete(root, key): if root is NULL return; recurse left or right until the node is found. Leaf: free it. One child: replace the node with its child. Two children: copy the inorder successor (minimum of the right subtree) into the node and delete that successor from the right subtree.
5. Display: perform an inorder traversal (left, root, right).
6. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
   int data;
   struct Node *left, *right;
};

/* Create a new node */
struct Node* createNode(int value) {
   struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = newNode->right = NULL;
   return newNode;
}

/* a) Insert an element into the BST */
struct Node* insert(struct Node* root, int value) {
   if (root == NULL)
      return createNode(value);
   if (value < root->data)
      root->left = insert(root->left, value);
   else if (value > root->data)
      root->right = insert(root->right, value);
   /* Duplicate values are ignored */
   return root;
}
```

```
/* Find the node with the minimum value (used in deletion) */
struct Node* findMin(struct Node* root) {
   while (root->left != NULL)
      root = root->left;
   return root;
}

/* b) Delete an element from the BST */
struct Node* deleteNode(struct Node* root, int key) {
   if (root == NULL)
      return root;
   if (key < root->data) {
      root->left = deleteNode(root->left, key);
   } else if (key > root->data) {
      root->right = deleteNode(root->right, key);
   } else {
      if (root->left == NULL) {
         struct Node* temp = root->right;
         free(root);
         return temp;
      } else if (root->right == NULL) {
         struct Node* temp = root->left;
         free(root);
         return temp;
      }
      struct Node* temp = findMin(root->right);
      root->data = temp->data;
      root->right = deleteNode(root->right, temp->data);
   }
   return root;
}

/* c) Search for a key in the BST */
struct Node* search(struct Node* root, int key) {
   if (root == NULL || root->data == key)
      return root;
   if (key < root->data)
      return search(root->left, key);
   return search(root->right, key);
}

/* Inorder traversal (prints elements in sorted order) */
void inorder(struct Node* root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

/* Menu-driven main function */
int main() {
   struct Node* root = NULL;
   int choice, value;

   do {
      printf("\n--- BST Operations Menu ---\n");
      printf("1. Insert\n");
      printf("2. Delete\n");
      printf("3. Search\n");
```

```
    printf("4. Display (Inorder)\n");
    printf("5. Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice) {
      case 1:
        printf("Enter value to insert: ");
        scanf("%d", &value);
        root = insert(root, value);
        break;
      case 2:
        printf("Enter value to delete: ");
        scanf("%d", &value);
        root = deleteNode(root, value);
        break;
      case 3:
        printf("Enter value to search: ");
        scanf("%d", &value);
        if (search(root, value) != NULL)
          printf("%d found in the BST.\n", value);
        else
          printf("%d not found in the BST.\n", value);
        break;
      case 4:
        printf("Inorder traversal: ");
        inorder(root);
        printf("\n");
        break;
      case 5:
        printf("Exiting program.\n");
        break;
      default:
        printf("Invalid choice. Try again.\n");
    }
  } while (choice != 5);

  return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
--- BST Operations Menu ---
1. Insert
2. Delete
3. Search
4. Display (Inorder)
5. Exit
Enter your choice: 1
Enter value to insert: 50
Enter your choice: 1
Enter value to insert: 30
Enter your choice: 1
Enter value to insert: 70
Enter your choice: 1
Enter value to insert: 20
Enter your choice: 4
Inorder traversal: 20 30 50 70
Enter your choice: 3
Enter value to search: 30
30 found in the BST.
Enter your choice: 2
Enter value to delete: 50
Enter your choice: 4
Inorder traversal: 20 30 70
Enter your choice: 5
Exiting program.
Data Structures Lab
```

# Program 12: BST – Create, Traverse and Search (Week 11)

## Aim

To design, develop and implement a menu-driven C program on a Binary Search Tree of integers to (a) create a BST of N integers: 13, 3, 4, 12, 14, 10, 5, 1, 8, 2, 7, 9, 11, 6, 18, (b) traverse the BST in inorder, preorder or postorder, (c) search the BST for a given element (KEY) and report an appropriate message, and (d) exit.

## Description

The BST is built by inserting the given elements one by one from an empty tree. The three depth-first traversals differ in when the root is visited: inorder (Left, Root, Right) gives sorted order, preorder (Root, Left, Right) visits the root first, and postorder (Left, Right, Root) visits the root last. Searching follows the BST ordering property, going left for smaller keys and right for larger keys, so it takes O(h) time where h is the height of the tree.

## Algorithm

1. Start. Declare the node structure and the array of the 15 given elements.
2. Create BST: set root = NULL and call insert(root, element) for every element of the array.
3. Inorder traversal: recursively traverse the left subtree, print the node, then traverse the right subtree.
4. Preorder traversal: print the node, then traverse the left subtree and the right subtree.
5. Postorder traversal: traverse the left subtree, the right subtree, then print the node.
6. Search: read the KEY; starting from root go left if KEY < data and right if KEY > data until the key is found or NULL is reached; print 'found' or 'not found'.
7. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>

// Structure for a BST node
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};

// Create a new node
struct Node* createNode(int value)
{
   struct Node *newNode;
   newNode = (struct Node*)malloc(sizeof(struct Node));
   newNode->data = value;
   newNode->left = NULL;
   newNode->right = NULL;
   return newNode;
}

// Insert an element into BST
struct Node* insert(struct Node *root, int value)
{
   if (root == NULL) {
      return createNode(value);
```

```
    }
    if (value < root->data)
    {
        root->left = insert(root->left, value);
    }
    else if (value > root->data)
    {
        root->right = insert(root->right, value);
    }
    return root;
}

// Inorder traversal
void inorder(struct Node *root)
{
    if (root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Preorder traversal
void preorder(struct Node *root)
{
    if (root != NULL)
    {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

// Postorder traversal
void postorder(struct Node *root)
{
    if (root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

// Search an element in BST
struct Node* search(struct Node *root, int key)
{
    if (root == NULL || root->data == key)
    {
        return root;
    }
    if (key < root->data)
    {
        return search(root->left, key);
    }
    else
    {
        return search(root->right, key);
    }
}
```

```
int main()
{
   struct Node *root = NULL;
   struct Node *result;
   int choice, key;

   // Given elements
   int elements[] = {
      13, 3, 4, 12, 14, 10, 5, 1,
      8, 2, 7, 9, 11, 6, 18
   };
   int n = 15;
   int i;

   do
   {
      printf("\n\n----- BINARY SEARCH TREE -----");
      printf("\n1. Create BST");
      printf("\n2. Inorder Traversal");
      printf("\n3. Preorder Traversal");
      printf("\n4. Postorder Traversal");
      printf("\n5. Search");
      printf("\n6. Exit");
      printf("\nEnter your choice: ");
      scanf("%d", &choice);

      switch (choice)
      {
         case 1:
            root = NULL;
            for (i = 0; i < n; i++)
            {
               root = insert(root, elements[i]);
            }
            printf("\nBST created successfully.");
            break;

         case 2:
            if (root == NULL)
            {
               printf("\nPlease create the BST first.");
            }
            else
            {
               printf("\nInorder Traversal: ");
               inorder(root);
            }
            break;

         case 3:
            if (root == NULL)
            {
               printf("\nPlease create the BST first.");
            }
            else
            {
               printf("\nPreorder Traversal: ");
               preorder(root);
            }
            break;
```

```
        case 4:
            if (root == NULL)
            {
                printf("\nPlease create the BST first.");
            }
            else
            {
                printf("\nPostorder Traversal: ");
                postorder(root);
            }
            break;

        case 5:
            if (root == NULL)
            {
                printf("\nPlease create the BST first.");
            }
            else
            {
                printf("\nEnter the element to search: ");
                scanf("%d", &key);
                result = search(root, key);
                if (result != NULL)
                {
                    printf("\nKEY %d is found in the BST.", key);
                }
                else
                {
                    printf("\nKEY %d is not found in the BST.", key);
                }
            }
            break;

        case 6:
            printf("\nExiting program...");
            break;

        default:
            printf("\nInvalid choice! Please try again.");
    }
  } while (choice != 6);

  return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
----- BINARY SEARCH TREE -----
1. Create BST
2. Inorder Traversal
3. Preorder Traversal
4. Postorder Traversal
5. Search
6. Exit
Enter your choice: 1
BST created successfully.
Enter your choice: 2
Inorder Traversal: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 18
Enter your choice: 3
Preorder Traversal: 13 3 1 2 4 12 10 5 8 7 6 9 11 14 18
Enter your choice: 4
Postorder Traversal: 2 1 6 7 9 8 5 10 11 12 4 3 18 14 13
Enter your choice: 5
Enter the element to search: 10
KEY 10 is found in the BST.
Enter your choice: 6
Exiting program...
Data Structures Lab
```

Dev-C++ console output | Sample run 12

# Program 13: BST – Non-Recursive Inorder, Preorder and Postorder Traversals (Week 12)

## Aim

To design, develop and implement a menu-driven C program to create a Binary Search Tree of the integers 6, 9, 5, 2, 8, 15, 24, 14, 7, 8, 5, 2 and to traverse it in inorder, preorder and postorder using non-recursive functions.

## Description

Recursive traversals use the system call stack implicitly; the non-recursive versions use an explicit stack of node pointers instead. The given data contains duplicates (8, 5, 2), which are ignored while constructing the BST, as is the usual BST convention. Inorder pushes the left spine and visits a node when it is popped; preorder pops a node, visits it and pushes its right then left child; postorder uses a lastVisited pointer so that a node is printed only after both its subtrees have been visited.

## Algorithm

1. Start. Declare the node structure, an explicit stack stack[MAX] and top = -1.
2. Create BST by inserting the 12 given values, ignoring duplicates.
3. Non-recursive Inorder: while current != NULL or the stack is not empty – push current and move left until NULL; pop a node, print it and move to its right child.
4. Non-recursive Preorder: push root; while the stack is not empty – pop a node, print it, push its right child (if any) and then its left child (if any).
5. Non-recursive Postorder: use current and lastVisited. Push nodes while moving left; when current is NULL look at the stack top – if it has a right child that is not yet visited move to it, otherwise print the node, set lastVisited and pop it.
6. Reset top = -1 before every traversal.
7. Repeat according to the menu choice until Exit. Stop.

## Program (C)

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

// Structure of a BST node
struct Node
{
   int data;
   struct Node *left;
   struct Node *right;
};

// Stack structure
struct Node *stack[MAX];
int top = -1;

// Push a node into stack
void push(struct Node *node)
{
   if (top == MAX - 1)
   {
      printf("Stack Overflow\n");
      return;
   }
```

```
    stack[++top] = node;
}

// Pop a node from stack
struct Node* pop()
{
    if (top == -1)
        return NULL;
    return stack[top--];
}

// Check whether stack is empty
int isEmpty()
{
    return top == -1;
}

// Create a new node
struct Node* createNode(int value)
{
    struct Node *newNode;
    newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

// Insert an element into BST
struct Node* insert(struct Node *root, int value)
{
    if (root == NULL)
        return createNode(value);
    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);
    // Duplicate values are ignored
    return root;
}

// Non-recursive Inorder Traversal
void inorder(struct Node *root)
{
    struct Node *current = root;
    while (current != NULL || !isEmpty())
    {
        // Move to the leftmost node
        while (current != NULL)
        {
            push(current);
            current = current->left;
        }
        // Visit the node
        current = pop();
        printf("%d ", current->data);
        // Move to right subtree
        current = current->right;
    }
}
```

```
// Non-recursive Preorder Traversal
void preorder(struct Node *root)
{
   struct Node *current;
   if (root == NULL)
      return;
   push(root);
   while (!isEmpty())
   {
      current = pop();
      printf("%d ", current->data);
      // Push right first
      if (current->right != NULL)
         push(current->right);
      // Push left next
      if (current->left != NULL)
         push(current->left);
   }
}

// Non-recursive Postorder Traversal
void postorder(struct Node *root)
{
   struct Node *current;
   struct Node *lastVisited = NULL;
   current = root;
   while (current != NULL || !isEmpty())
   {
      if (current != NULL)
      {
         push(current);
         current = current->left;
      }
      else
      {
         current = stack[top];
         if (current->right != NULL &&
            lastVisited != current->right)
         {
            current = current->right;
         }
         else
         {
            printf("%d ", current->data);
            lastVisited = current;
            pop();
            current = NULL;
         }
      }
   }
}

int main()
{
   struct Node *root = NULL;
   int elements[] = {
      6, 9, 5, 2, 8, 15,
      24, 14, 7, 8, 5, 2
   };
   int n = 12;
   int choice;
   int i;
```

```
do
{
   printf("\n\n----- BINARY SEARCH TREE -----");
   printf("\n1. Create BST");
   printf("\n2. Inorder Traversal");
   printf("\n3. Preorder Traversal");
   printf("\n4. Postorder Traversal");
   printf("\n5. Exit");
   printf("\nEnter your choice: ");
   scanf("%d", &choice);

   switch (choice)
   {
      case 1:
         root = NULL;
         for (i = 0; i < n; i++)
         {
            root = insert(root, elements[i]);
         }
         printf("\nBST created successfully.");
         break;

      case 2:
         if (root == NULL)
         {
            printf("\nPlease create the BST first.");
         }
         else
         {
            top = -1;
            printf("\nInorder Traversal: ");
            inorder(root);
         }
         break;

      case 3:
         if (root == NULL)
         {
            printf("\nPlease create the BST first.");
         }
         else
         {
            top = -1;
            printf("\nPreorder Traversal: ");
            preorder(root);
         }
         break;

      case 4:
         if (root == NULL)
         {
            printf("\nPlease create the BST first.");
         }
         else
         {
            top = -1;
            printf("\nPostorder Traversal: ");
            postorder(root);
         }
         break;
```

```
        case 5:
            printf("\nExiting program...");
            break;

        default:
            printf("\nInvalid choice! Please try again.");
    }
  } while (choice != 5);

  return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
----- BINARY SEARCH TREE -----
1. Create BST
2. Inorder Traversal
3. Preorder Traversal
4. Postorder Traversal
5. Exit
Enter your choice: 1
BST created successfully.
Enter your choice: 2
Inorder Traversal: 2 5 6 7 8 9 14 15 24
Enter your choice: 3
Preorder Traversal: 6 5 2 9 8 7 15 14 24
Enter your choice: 4
Postorder Traversal: 2 5 7 8 14 24 15 9 6
Enter your choice: 5
Exiting program...
Data Structures Lab
```

Dev-C++ console output | Sample run 13

# Program 14: Graph of N Cities – BFS and DFS (Program 13)

## Aim

To create a graph of N cities using an adjacency matrix and to print all nodes reachable from a given starting node in a digraph using Breadth First Search (BFS) and Depth First Search (DFS).

## Description

In the adjacency matrix representation adj[i][j] = 1 if there is an edge from city i to city j, otherwise 0; for a digraph the matrix need not be symmetric. BFS visits nodes level by level using a queue, while DFS goes as deep as possible before backtracking, using recursion (the call stack). Both algorithms take $O(V^2)$ time with an adjacency matrix.

## Algorithm

1. Start. Read the number of cities n, the adjacency matrix and the starting node s.
2. BFS: mark s visited and enqueue it. While the queue is not empty, dequeue u, print it and enqueue every unvisited v with adj[u][v] == 1 (marking it visited).
3. DFS: mark u visited and print it; for every v with adj[u][v] == 1 and v unvisited, call dfs(v).
4. Reset the visited[] array between the two traversals.
5. Print the nodes reachable using BFS and using DFS. Stop.

## Program (C)

```
#include <stdio.h>
#define MAX 20

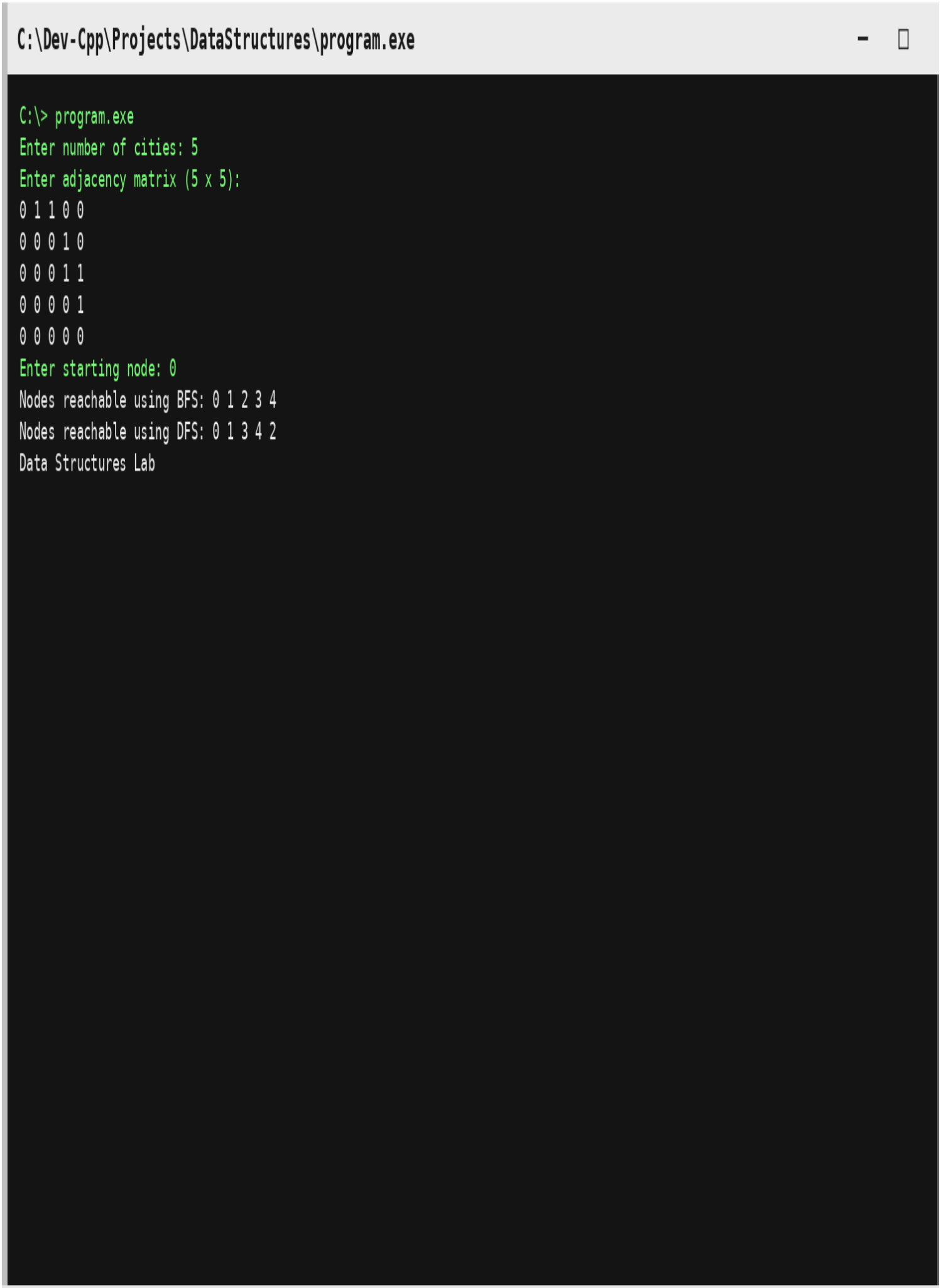
int n, adj[MAX][MAX], visited[MAX];

void bfs(int s)
{
   int q[MAX], f = 0, r = 0, u, v;
   for (v = 0; v < n; v++) visited[v] = 0;
   visited[s] = 1;
   q[r++] = s;
   while (f < r) {
      u = q[f++];
      printf("%d ", u);
      for (v = 0; v < n; v++)
         if (adj[u][v] && !visited[v]) {
            visited[v] = 1;
            q[r++] = v;
         }
   }
}

void dfs(int u)
{
   int v;
   visited[u] = 1;
   printf("%d ", u);
   for (v = 0; v < n; v++)
      if (adj[u][v] && !visited[v])
         dfs(v);
}

int main()
{
```

```
    int i, j, s;
    printf("Enter number of cities: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (%d x %d):\n", n, n);
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            scanf("%d", &adj[i][j]);
    printf("Enter starting node: ");
    scanf("%d", &s);
    printf("Nodes reachable using BFS: ");
    bfs(s);
    for (i = 0; i < n; i++) visited[i] = 0;
    printf("\nNodes reachable using DFS: ");
    dfs(s);
    printf("\n");
    return 0;
}
```




Dev-C++ console output | Sample run 14

# Program 15: All-Pairs Shortest Paths – Warshall's / Floyd–Warshall (Program 14)

## Aim

To find the shortest distance between every pair of vertices in an edge-weighted directed graph, represented as an adjacency (cost) matrix, using Warshall's (Floyd–Warshall) algorithm.

## Description

Plain Warshall's algorithm computes the transitive closure; with edge weights it becomes the Floyd–Warshall dynamic programming algorithm. For each intermediate vertex k it checks whether going i → k → j is shorter than the current i → j path, using d[i][j] = min(d[i][j], d[i][k] + d[k][j]). A missing edge is stored as INF = 10000000, which is large enough to act as infinity while the sum of two INF values still does not overflow an int. Time complexity is $O(V^3)$ and space complexity is $O(V^2)$.

## Algorithm

1. Start. Read the number of vertices n and the cost matrix d[][] (use 10000000 for INF).
2. For k = 0 to n-1 (intermediate vertex): for every pair (i, j) if d[i][k] + d[k][j] < d[i][j] then set d[i][j] = d[i][k] + d[k][j].
3. After all values of k have been processed, d[i][j] holds the shortest distance from i to j.
4. Print the matrix, showing INF where no path exists. Stop.

## Program (C)

```
#include <stdio.h>
#define MAX 20
#define INF 10000000

int n, d[MAX][MAX];

void floyd()
{
   int i, j, k;
   for (k = 0; k < n; k++)
      for (i = 0; i < n; i++)
         for (j = 0; j < n; j++)
            if (d[i][k] + d[k][j] < d[i][j])
               d[i][j] = d[i][k] + d[k][j];
}

int main()
{
   int i, j;
   printf("Enter number of vertices: ");
   scanf("%d", &n);
   printf("Enter cost matrix (use 10000000 for INF):\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         scanf("%d", &d[i][j]);
   floyd();
   printf("All-pairs shortest distance matrix:\n");
   for (i = 0; i < n; i++) {
      for (j = 0; j < n; j++) {
         if (d[i][j] >= INF) printf("%6s", "INF");
         else printf("%6d", d[i][j]);
      }
      printf("\n");
```

```
    }
    return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
Enter number of vertices: 4
Enter cost matrix (use 10000000 for INF):
0 5 10000000 10
10000000 0 3 10000000
10000000 10000000 0 1
10000000 10000000 10000000 0
All-pairs shortest distance matrix:
     0     5     8     9
   INF     0     3     4
   INF   INF     0     1
   INF   INF   INF     0
Data Structures Lab
```

Dev-C++ console output | Sample run 15

# Program 16: Dijkstra's Shortest Path – Start to End Node (Program 15)

## Aim

To find the shortest distance (and path) from a start node to an end node in a weighted graph using Dijkstra's algorithm.

## Description

Dijkstra's algorithm is a greedy algorithm that works only for non-negative edge weights. It repeatedly picks the unvisited vertex with the smallest tentative distance and relaxes its neighbours: if dist[u] + w(u,v) < dist[v], dist[v] is updated. The prev[] array stores each node's predecessor so that the path can be printed by walking back from the destination. With an adjacency matrix the time complexity is $O(V^2)$. In this program a weight of 0 means 'no edge'.

## Algorithm

1. Start. Read the number of nodes n, the cost matrix (0 = no edge), and the start and end nodes.
2. Initialise dist[i] = INF, prev[i] = -1 and visited[i] = 0 for all nodes; set dist[src] = 0.
3. Repeat n times: pick the unvisited node u with the smallest dist (stop if none is reachable); mark u visited.
4. Relax every neighbour v of u that is unvisited: if dist[u] + cost[u][v] < dist[v] then update dist[v] and set prev[v] = u.
5. If dist[dest] is still INF print 'No path'; otherwise print dist[dest].
6. Print the path by walking back from dest through prev[] until -1 and printing it in reverse. Stop.

## Program (C)

```
#include <stdio.h>
#define MAX 20
#define INF 10000000

int n, cost[MAX][MAX];

void dijkstra(int src, int dest)
{
   int dist[MAX], prev[MAX], visited[MAX];
   int i, count, u, v, min;

   for (i = 0; i < n; i++) {
      dist[i] = INF;
      prev[i] = -1;
      visited[i] = 0;
   }
   dist[src] = 0;

   for (count = 0; count < n; count++) {
      min = INF; u = -1;
      for (i = 0; i < n; i++)
         if (!visited[i] && dist[i] < min) {
            min = dist[i];
            u = i;
         }
      if (u == -1) break;        /* remaining nodes unreachable */
      visited[u] = 1;
      for (v = 0; v < n; v++)
         if (cost[u][v] && !visited[v] &&
```

```
            dist[u] + cost[u][v] < dist[v]) {
            dist[v] = dist[u] + cost[u][v];
            prev[v] = u;
         }
   }

   if (dist[dest] == INF) {
      printf("No path from %d to %d\n", src, dest);
      return;
   }
   printf("Shortest distance from %d to %d = %d\n", src, dest, dist[dest]);

   /* print path by walking back through prev[] */
   int path[MAX], len = 0;
   for (v = dest; v != -1; v = prev[v])
      path[len++] = v;
   printf("Path: ");
   for (i = len - 1; i >= 0; i--) {
      printf("%d", path[i]);
      if (i) printf(" -> ");
   }
   printf("\n");
}

int main()
{
   int i, j, src, dest;
   printf("Enter number of nodes: ");
   scanf("%d", &n);
   printf("Enter cost matrix (0 = no edge):\n");
   for (i = 0; i < n; i++)
      for (j = 0; j < n; j++)
         scanf("%d", &cost[i][j]);
   printf("Enter start node and end node: ");
   scanf("%d %d", &src, &dest);
   dijkstra(src, dest);
   return 0;
}
```

C:\Dev-Cpp\Projects\DataStructures\program.exe

```
C:\> program.exe
Enter number of nodes: 5
Enter cost matrix (0 = no edge):
0 10 0 30 0
0 0 50 0 0
0 0 0 0 10
0 0 20 0 60
0 0 0 0 0
Enter start node and end node: 0 4
Shortest distance from 0 to 4 = 40
Path: 0 -> 3 -> 2 -> 4
Data Structures Lab
```

Dev-C++ console output | Sample run 16